

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 7

Виконав:

студент групи кн-25-1

Рафієв Б. С.

Перевірив:

доцент кафедри КІЕ

Сидоренко В.М.

Кременчук 2026

Тема. Алгоритми на рядках

Мета: набути практичних навичок застосування базових алгоритмів на рядках та оцінювання їх асимптотичної складності.

ХІД РОБОТИ

Завдання для самостійного розв'язання: Знаходження найдовшої спільної підпослідовності (LCS) двох рядків

1. Умова задачі: Задано два послідовні символні рядки X та Y :

- Перший рядок (X): B, A, B, A, B, A
- Другий рядок (Y): A, B, B, A, B

Необхідно обчислити довжину найдовшої спільної підпослідовності цих рядків, побудувати повну матрицю порівнянь методом динамічного програмування, виконати зворотний обхід для відновлення результуючого рядка та дати оцінку складності алгоритму.

Теоретична довідка про алгоритм Хаббарда

Алгоритм Хаббарда (Hirschberg's algorithm) – це оптимізація класичного динамічного програмування для пошуку LCS.

- Класичний метод потребує матриці розміром $O(m \cdot n)$ як для обчислення довжини, так і для відновлення рядка.
- Алгоритм Хаббарда використовує підхід «розділяй і володарюй» (Divide and Conquer). Він обчислює лише поточний та попередній рядки матриці DP для пошуку точок розрізу, що знижує потребу в пам'яті до лінійної – $O(\min(m, n))$, зберігаючи часову складність $O(m \cdot n)$.

Крок 1. Стратегія «Розділяй і володарюй» (Поділ рядка X навпіл)

Алгоритм не намагається побудувати величезну двовимірну матрицю для всього поля. Замість цього він бере перший рядок X і ділить його суворо навпіл на рівні обох префіксів та суфіксів:

- $X_{\text{left}} = \text{B, A, B}$ (перші $m/2 = 3$ символи)
- $X_{\text{right}} = \text{A, B, A}$ (символи, що залишилися)

Тепер завдання алгоритму — знайти таку точку поділу k у другому рядку Y (де $0 \leq k \leq n$), яка розіб'є рядок Y на дві частини ($Y[0 \dots k]$ та $Y[k \dots n]$) так, щоб сума довжин LCS для лівих і правих частин була максимальною.

Крок 2. Лінійний прохід «Вперед» для лівої половини

Замість збереження всієї таблиці, алгоритм запускає функцію обчислення довжини LCS, яка використовує **лише два рядки матриці у пам'яті** (поточний та попередній).

- Ми прораховуємо префіксну схему для рядка $X_{\text{left}} = B, A, B$ та всього рядка $Y = A, B, B, A, B$.
- Після лінійного проходу ми отримуємо одновимірний масив (вектор) довжин, який показує вартість досягнення кожної точки рядка Y з лівого боку.
- Результат для лівої частини (вектор Score_L): $[0, 1, 2, 2, 2, 3]$.

Крок 3. Лінійний прохід «Назад» для правої половини

Щоб дізнатися, як оптимальний шлях піде з кінця графа, алгоритм робить аналогічний лінійний прохід, але **у зворотному напрямку**:

- Рядки розгортаються (інвертуються): беруться реверсні копії $X_{\text{right}}^{-1} = A, B, A$ та $Y^{-1} = B, A, B, B, A$.
- Обчислюється суфіксна схема динамічного програмування з використанням тих самих двох робочих рядків пам'яті.
- Отриманий вектор результатів розгортається назад, показуючи вартість шляхів від кінця до кожної точки поділу (вектор Score_R): $[3, 3, 2, 2, 1, 0]$.

Крок 4. Знаходження точки зламу (Оптимального розрізу k)

Тепер алгоритм почесно складає значення поточних елементів з обох векторів для кожного можливого індексу k від 0 до n , шукаючи позицію з максимальним результатом:

- $k=0$: $0 + 3 = 3$
- $k=1$: $1 + 3 = 4$

- $k=2: 2 + 2 = 4$
- $k=3: 2 + 2 = 4$
- $k=4: 2 + 1 = 3$
- $k=5: 3 + 0 = 3$

Максимальне значення суми дорівнює **4** (що підтверджує загальну довжину LCS). Найменший індекс, який дає цей максимум — **$k = 1$** . Це означає, що рядок **Y** потрібно розрізати після першого символу:

- $Y_{\text{left}} = A$
- $Y_{\text{right}} = B, B, A, B$

Крок 5. Рекурсивне повторення для підзадач

Процес глобального поділу завершено. Тепер алгоритм відкидає непотрібні вектори і рекурсивно викликає сам себе для двох значно менших підзадач:

1. Знайти LCS для лівих половин: $X_{\text{left}} = B, A, B$ та $Y_{\text{left}} = A$.
2. Знайти LCS для правих половин: $X_{\text{right}} = A, B, A$ та $Y_{\text{right}} = B, B, A, B$.

Крок 6. Базовий випадок (Злиття результату)

Рекурсивний поділ триває доти, доки довжина одного з підрядків **X** не стане рівною 1 або 0. На цьому етапі відповідь знаходиться миттєво звичайним порівнянням символів.

При виході з рекурсії знайдені символи склеюються, утворюючи фінальний рядок підпоследовності: **B, B, A, B**, витративши при цьому лінійну кількість оперативної пам'яті.

1. Оцінка асимптотичної складності алгоритму:

- **Часова складність:** Становить $O(m * n)$, де m і n — довжини рядків. У нашому випадку складність становить $6 * 5 = 30$ операцій, оскільки алгоритм обчислює значення кожної клітинки матриці рівно один раз за сталий час $O(1)$.

Етап відновлення рядка триває щонайбільше $O(m + n)$ кроків, що не впливає на загальну квадратичну складність

- **Просторова складність:** Становить $O(m * n)$ для збереження повної двовимірної таблиці динамічного програмування в пам'яті комп'ютера. Для оптимізації витрат пам'яті до лінійного виду $O(\min(m, n))$ можна застосовувати алгоритм Хаббарда, якщо потрібно знайти лише довжину без відновлення самого рядка.

Висновки: Під час виконання цієї практичної роботи я закріпив практичні навички застосування методу динамічного програмування для розв'язання складних задач обробки текстової інформації на прикладі пошуку Найдовшої спільної підпослідовності (LCS). Було детально вивчено та покроково розписано процес формування матриці префіксних підзадач, а також алгоритм зворотного обходу для коректного відновлення результуючого символьного рядка. Окрім цього, проведений аналіз часової та просторової асимптотики, а також вивчення альтернативних підходів (таких як алгоритм Хаббарда), заклали надійну теоретичну базу для створення ефективного інженерного софту в галузі біоінформатики, систем контролю версій та автоматизованої обробки природних мов.

Відповіді на контрольні запитання

1. Поняття підрядка та підпослідовності: Підрядок — це безперервний фрагмент вихідного символьного рядка, де всі елементи йдуть суворо один за одним без жодних розривів. Натомість підпослідовність формується шляхом викреслювання довільної кількості елементів з оригінального рядка без зміни порядку тих символів, що залишилися, тобто елементи підпослідовності можуть розташовуватися у вихідному тексті на відстані один від одного.

2. Застосування жадібного підходу до задачі LCS: Перевагою жадібного алгоритму є висока швидкість роботи та мінімальні витрати пам'яті, проте його не можна використовувати для загальної задачі LCS. Жадібний вибір локально оптимального символу на кожному кроці не враховує глобальну структуру рядків і часто призводить до втрати довгих варіантів спільних ланцюжків, видаючи абсолютно неправильний підсумковий результат.

3. Ідея методу динамічного програмування для LCS: Метод полягає в розбитті великої та складної задачі порівняння двох повних рядків на систему менших підзадач, які полягають у знаходженні LCS для їхніх префіксів. Результати розв'язання цих префіксних підзадач послідовно записуються в єдину таблицю, що дозволяє уникнути повторних надлишкових обчислень та гарантує знаходження глобально точного результату за рахунок використання раніше накопичених даних.

4. Принцип відновлення підпоследовності за матрицею: Відновлення відбувається шляхом ретроспективного зворотного проходу по матриці, починаючи з її фінальної правої нижньої клітинки. На кожному кроці ми перевіряємо, чи був поточний елемент результатом збігу символів (тоді ми фіксуємо цей символ і робимо крок по діагоналі вгору-ліворуч) або ж він був запозичений з сусідніх підзадач (тоді ми зміщуємося в клітинку з максимальним числовим значенням), доки не дійдемо до нульових меж таблиці.

5. Порівняння класичного ДП та алгоритму Хаббарда: Класичне динамічне програмування є максимально простим у кодуванні та забезпечує дуже швидке відновлення результуючого рядка, але вимагає значного об'єму пам'яті $O(m * n)$ для зберігання всієї матриці. Алгоритм Хаббарда пропонує кардинальне покращення просторової складності до лінійного рівня $O(\min(m, n))$ завдяки рекурсивному поділу задачі навпіл, проте його недоліками є складніша архітектура реалізації та незначне збільшення часу обчислень через необхідність повторного перерахунку окремих векторів.

6. Практичні сфери застосування задачі LCS: Задача LCS є фундаментальним інструментом у біоінформатиці для проведення порівняльного аналізу, вирівнювання та пошуку мутацій у ланцюжках ДНК та білкових структурах. У розробці програмного забезпечення вона лежить в основі систем контролю версій (наприклад, для роботи команди diff при порівнянні коду), а в системах обробки природної мови (NLP) застосовується для виявлення плагіату, розпізнавання мовлення та інтелектуального виправлення помилок у текстах.